FACE MASK DETECTION USING AI AND IOT A PROJECT REPORT Submitted By BACHELOR OF ENGINEERING IN ELECTRONICS AND COMMUNICATION ENGINEERING INSTITUTE OF ROAD AND TRANSPORT TECHNOLOGY,...

Thesis · June 2021

DOI: 10.13140/RG.2.2.23151.15521

4 authors, including:

Dr R Senthilkumar
Institute of Road and Transport Technology (IRTT)
89 PUBLICATIONS **135** CITATIONS

SEE PROFILE

FACE MASK DETECTION USING AI AND IOT

A PROJECT REPORT

Submitted By

JAYARAGUL KAN A.R	**Reg.No: 731117106017**
MANO MU	**Reg.No: 731117106027**
SARAN PRAKASH M	**Reg.No: 731117106042**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

INSTITUTE OF ROAD AND TRANSPORT TECHNOLOGY, ERODE

ANNA UNIVERSITY : CHENNAI 600 025

APRIL 2021

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report **"FACE MASK DETECTION USING AI AND IOT"** is the bonafide work of

JAYARAGUL KAN A.R **Reg.No: 731117106017**

MANO MU **Reg.No: 731117106027**

SARAN PRAKASH M **Reg.No: 731117106042**

who carried out the project work under my supervision.

SIGNATURE

Dr. VALARMATHI R
HEAD OF THE DEPARTMENT
Professor
ECE,
IRTT, Erode.

SIGNATURE

Dr. SENTHIL KUMAR R
SUPERVISOR
Assistant Professor
ECE,
IRTT, Erode.

Submitted for the Anna University examination held on _____

at Institute of Road and Transport Technology (IRTT), Erode.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We sincerely express our whole hearted thanks to the principal Dr. R. MURUGESAN M.E., Ph. D., Institute of Road and Transport Technology, Erode for his constant encouragement and moral support during the course of this project.

We owe our sincere thanks to Dr. R. VALARMATHI ME., Ph. D., Head of the Department of Electronics and Communication Engineering, Institute of Road and Transport Technology, Erode, for furnishing every essential facility for doing this project.

We sincerely thank our guide Dr. R. SENTHIL KUMAR M.E., Ph.D., Assistant Professor, Department of Electronics and Communication Engineering, Institute of Road and Transport Technology, Erode, for his valuable help and guidance throughout the project.

We wish to express our sincere thanks to the project coordinator and all staff members, Department of Electronics and Communication Engineering for their valuable help and guidance rendered to us throughout the project.

Above all we are grateful to all our classmates and friends for their friendly cooperation and their exhilarating company.

NOTE:

The complete report was prepared in "LATEX" document preparation software.

Abstract

The novel Coronavirus had brought a new normal life in which the social distance and wearing of face masks plays a vital role in controlling the spread of virus. But most of the people are not wearing face masks in public places which increases the spread of viruses. This may result in a serious problem of increased spreading. Hence to avoid such situations we have to scrutinize and make people aware of wearing face masks. Humans cannot be involved for this process, due to the chance of getting affected by corona.

Hence here comes the need for artificial intelligence(AI), which is the main theme of our project. Our project involves the identification of persons wearing face masks and not wearing face masks in public places by means of using image processing and AI techniques and sending alert messages to authority persons. The object detection algorithms are used for identification of persons with and without wearing face masks which also gives the count of persons wearing mask and not wearing face mask and Internet Of Things (IOT) is utilized for sending alert messages. The alert messages are sent to the authority persons through mobile notification and Email. Based on the count of persons wearing and not wearing face masks the status is obtained. Depending upon the status warning is done by means of using buzzer and LED's.

Contents

List of Figures

Chapter 1

INTRODUCTION

The novel coronavirus covid-19 had brought a new normal life.India is struggling to get out of this virus attack and the government implemented lockdown for the long way. Lockdown placed a pressure on the global economy. So the government gave relaxations in lockdown . Declared by the WHO that a potential speech by maintaining distance and wearing a mask is necessary. The biggest support that the government needs after relaxation is social distancing and wearing of masks by the people.But many people are getting out without a face mask this may increase the spread of covid-19. Economic Times India has stated that " Survey Shows that 90 percent Indians are aware, but only 44 percent wearing a mask ". This survey clearly points that people are aware but they are not wearing the mask due to some discomfort in wearing and carelessness.This may result in the easy spreading of covid-19 in public places.

The world health organisation has clearly stated that until vaccines are found the wearing of masks and social distancing are key tools to reduce spread of virus.So it is important to make people wear masks in public places. In densely populated regions it is difficult to find the persons not wearing the face mask and warn them.Hence we are using image processing techniques for identification of persons wearing and not wearing face masks. In real time images are collected from the camera and it is processed in Raspberry Pi embedded development kit. The real time images from the camera are compared with the trained dataset and detection of wearing or

not wearing a mask is done. The trained dataset is made by using machine learning technique which is the deciding factor of the result.The algorithm created by means of using a trained dataset will find the persons with and without wearing face masks.

The Internet of Things (IOTs) can be used for connecting objects like smartphones, Internet TVs, laptops, computers, sensors and actuators to the Internet where the devices are linked together to enable new forms of communication between things and people, and between things themselves. Intimation messages are sent to authority persons by means of using IOT .

The Chapter 2 explains in detail the step by step procedure of the proposed algorithm. The experimental setup of the proposed hardware model is explained in detail in Chapter 3. The experimental results obtained are briefly plotted in Chapter 4. Chapter 5 concludes the proposed method performance and its related future work.

Chapter 2

LITERATURE REVIEW

2.1 Introduction

The literature review is split into three main categories. In the first category, the literature related to image classification using deep learning techniques is discussed. In the second category, the Internet of Things (IOT) concepts are discussed. In the third category, the literature related to combined IOT devices and deep learning techniques are discussed briefly here.

2.2 Machine learning for image classification

In the content based image classification using deep learning , Joseph Redmon et.al proposed You Only Look Once (YOLO) algorithm for real time object detection.

Sanzidul Islam et.al 2020, gave a deep learning based assistive System to classify COVID-19 Face Mask which is implemented in rasbperrypi-3.

Velantina et.al 2020, made an COVID-19 facemask detection by means of using Caffe model.

Senthilkumar et.al 2017, compared the two most frequently used machine learning algorithms K-Nearest Neighbour and Support Vector Machine in his work for face recognition.

Senthilkumar et.al 2018, proposed a new and fast approach for face recognition.

2.3 Internet of Things

Luigi Atzori et.al reviewed different versions of the Internet of Things are reported and corresponding enabling technologies.

Lu Tan et.al and Neng Wang discussed the Future internet in their work.

Feng Xia et.al and others discussed briefly about the Internet of Things, 2012 in their work.

2.4 IOT device and Machine Learning

Yair Meidan et.al 2017, has implemented nine IOT devices and treated each IOT device as separate classes. For classification purposes, deep learning techniques were used.

Yair Meidan et.al and Michael Bohadana et.al 2017, proposed a security system for detection of unauthorized IoT devices using machine learning techniques.

Liang Xiao et.al 2018, has improved the IoT Security techniques based on machine learning using Artificial Intelligence concept.

With this, based on the above literature surveys, we have made a new deep learning algorithm for face mask detection. The details are elaborated in forthcoming chapters.

Chapter 3

FACE MASK DETECTION ALGORITHM DEVELOPMENT

In this chapter, the algorithm for detection of persons with face masks is discussed in detail. YOLO object detection algorithm is used for detection of persons with face mask and without face mask. Here yolo workflow is discussed in step by step.

3.1 YOLO - object detection algorithm

Deep Learning consists of a very enormous number of neural networks that use the multiple cores of a process of a computer and video processing cards to manage the neuranetwork's neuron which is categorized as a single node. Deep learning is used in numerous applications because of its popularity especially in the field of medicine and agriculture. Here YOLO deep learning technique is used to identify persons wearing and not wearing face masks.

Joseph Redmon et al. introduced You look only once also known as YOLO in 2015. YOLO is a convolutional neural network (CNN) for doing object detection in real-time. The algorithm applies a single neural net-

work to the full image, and then divides the image into regions and predicts bounding boxes and probabilities for each region. These bounding boxes are weighted by the predicted probabilities. Some improvements were done over years and YOLOv2 and YOLOv3 versions were introduced respectively in 2016 , 2018.Our model uses YOLOv3 and it provides good results regarding object classification and detection. In the previous version of Yolov2 Darknet-19 is used. Yolov3 uses darknet-53. Darknet is a framework used for training neural networks written in C language.

3.2 Benefits

YOLO is a popular object detection algorithm because it achieves high accuracy while it is also able to run in real-time. The algorithm "only looks once" at the image means that it requires only one forward propagation pass through the neural network to make predictions. After non-max suppression it then gives the recognized objects along with the bounding boxes. In YOLO, a single CNN simultaneously predicts multiple bounding boxes and class probabilities for those boxes. YOLO directly optimizes detection performance since it trains on full images. YOLO has a number of benefits over other object detection methods they are

- YOLO is extremely fast
- YOLO scans the entire image during training and also during testing. So, it implicitly encodes contextual information about classes as well as their appearance.
- YOLO learns generalizable representations of objects so that when it is trained on natural images and tested , the algorithm performs excellently when compared to other top detection methods.

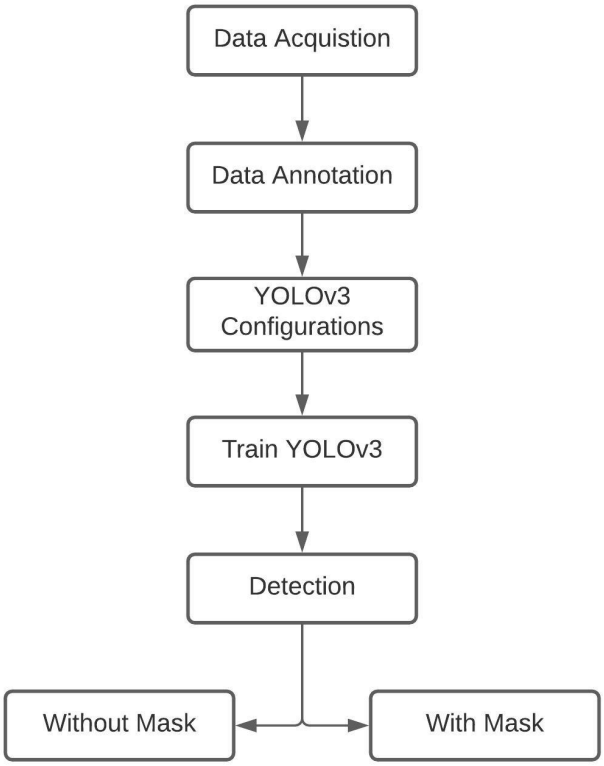


Figure 3.1: YOLO work flow.

3.3 Workflow

Here the workflow of YOLO object detection algorithm is discussed in detail. Initially a dataset of images is collected which are used for training by means of using YOLO. Dataset consists of images of persons with masks and without masks. figure 3.1 shows the work flow of YOLO.

3.3.1 Data Acquisition

Data is really important for deep learning techniques. If we use more data for training the AI then the result will be better. To train YOLO we need more data and with proper annotation. Using a web-scraping tool we have collected 900 images of both mask and no-mask. These images cannot be used directly so we need to pre-process before feeding into the model. Next step is Data Annotation.

3.3.2 Data Annotation

To train YOLO we need to annotate images for object detection models. Our dataset should be well annotated. There are different types of annotations available. Here a bounding boxes method is used. It creates a rectangle area over images that are present in our dataset. Since Annotation needs more time we are using a tool called LabelIMG to annotate our data.

3.3.3 YOLOv3 Configuration

The YOLOv3 configuration involved the creation of two files and a custom Yolov3 cfg file. YOLOv3 configuration first creates a ”obj.names” file which contains the name of the classes which the model wanted to detect. Then a obj.data file which contains a number of classes in here is 2, train data directory, validation data, ”obj.names” and weights path which is saved on the backup folder. Lastly, a cfg file contains 2 classes. figure 3.2 shows the configuration steps involved. Next is training of our YOLOv3 in which an input image is passed into the YOLOv3 model. This will go through the image and find the coordinates that are present. It divides the image into a grid and from that grid it analyzes the target objects features. Here 80 percent data is used for training , and remaining 20 percent is used for validation. Now weights of YOLOv3 trained on the dataset are created under a file. Using these trained weights now we can classify the persons wearing and not wearing the mask.

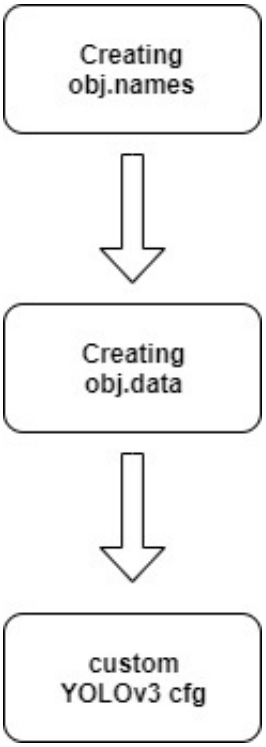


Figure 3.2: YOLOv3 Configuration.

3.3.4 Face Mask Detection Algorithm

Step 1: Start the program.

Step 2: Input image is feeded.

Step 3: YOLOv3 trained weights are loaded from the disk.

Step 4: Persons with and without face mask are detected by means of object detection algorithm.

Step 5: After detection resultant image is displayed along with count of Persons with and without masks.

Step 6: The ratio of with and without face mask is calculated and based upon ratio status is obtained.

Step 7: Based on status output LED and buzzer connected to Raspberry pi will be activated.

Step 8: Resultant image is saved in Raspberry pi for identification.

Chapter 4

EXPERIMENTAL SETUP FOR PROPOSED MODEL

In this chapter, first the block diagram representation of the proposed model is shown. Then the installation IOT server app in android mobile phone and its configuration for our project work is explained. In the third step, the configuration of Raspberry Pi embedded system GPIO ports and its hardware interface such as switch, LED and buzzer. As a final step, the interfacing of Raspberry Pi and its accessories with Laptop configured with VNC viewer.

4.1 Block diagram

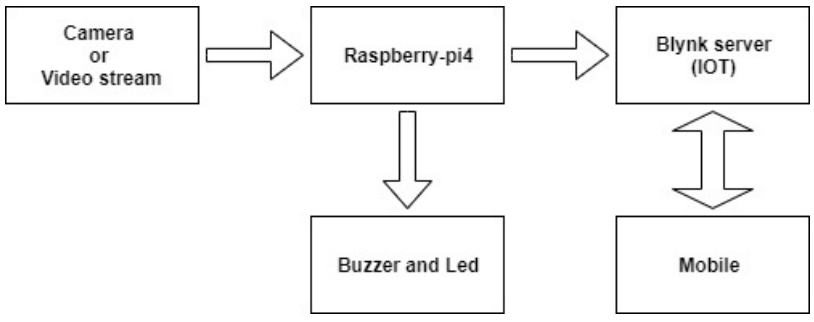


Figure 4.1: Block diagram of proposed model.

4.2 Hardware description

Raspberry Pi4 Development kit is used in this face mask detection system. The Raspberry Pi is a very cheap computer which runs on Linux OS, but it also provides a set of GPIO pins that are used to control electronic components and also Internet of Things . It is also used for image processing projects because of its processing speed and size. figure 4.1 shows the model setup of our project.Either camera or video stream is used as an input.The raspberry pi is connected with a buzzer and indication leads. For communication purposes we are using Blynk server (IOT) which is connected with the raspberry pi. figure 4.2 shows the Raspberry Pi 4 pin details.

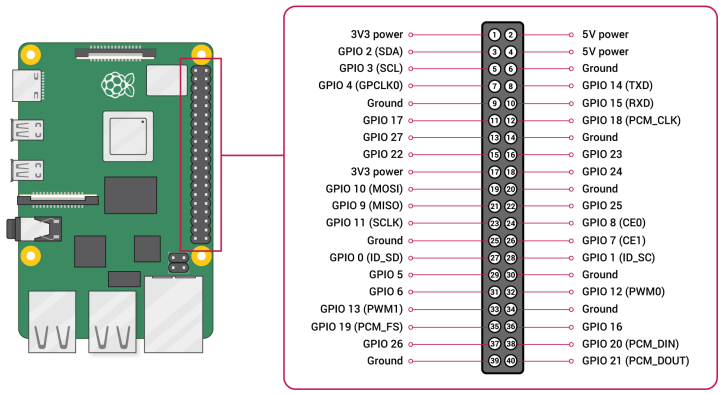


Figure 4.2: Raspberry Pi 4 pin details.

Blynk Server is used for sending messages between Blynk mobile applications and various development kits. Blynk IOT server is installed in this system and an activation link sent to registered email id. This activation link is copied and pasted in the Raspberry Pi terminal in order to interconnect this Raspberry Pi and Blynk IOT server. figure 4.3 is the images of the blynk server mobile app and figure 4.4 shows connections established between raspberry pi and Blynk server.

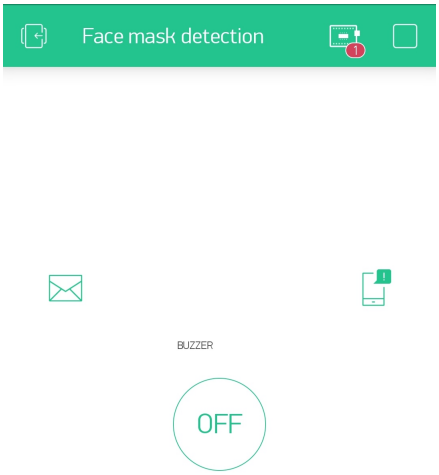


Figure 4.3: Blynk Mobile App Interface.

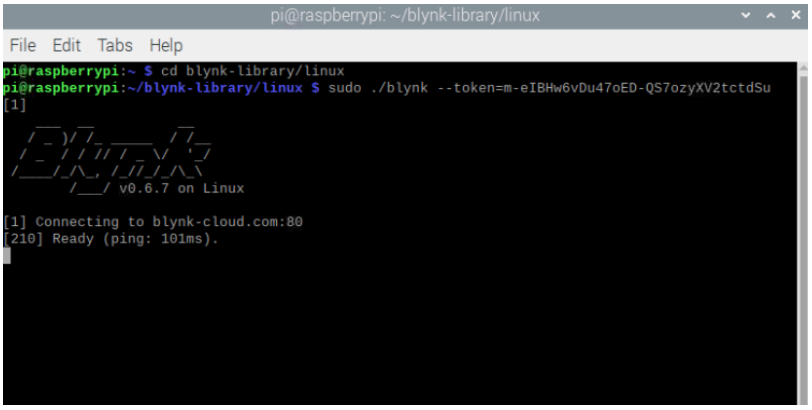


Figure 4.4: Blynk connected in Raspberry Pi.

The LED's and Buzzer are connected to raspberry pi by means of a general I/O pins of raspberry pi. There are two LED's one is of green colour and the other is of red colour. The buzzer used here is a 5v buzzer. figure 4.5 shows the prototype model. The Raspberry Pi can be used as a standalone computer by means of connecting peripherals such as monitor, keyboard. Here in this project we have connected Raspberry Pi with laptop by means of using VNC viewer.

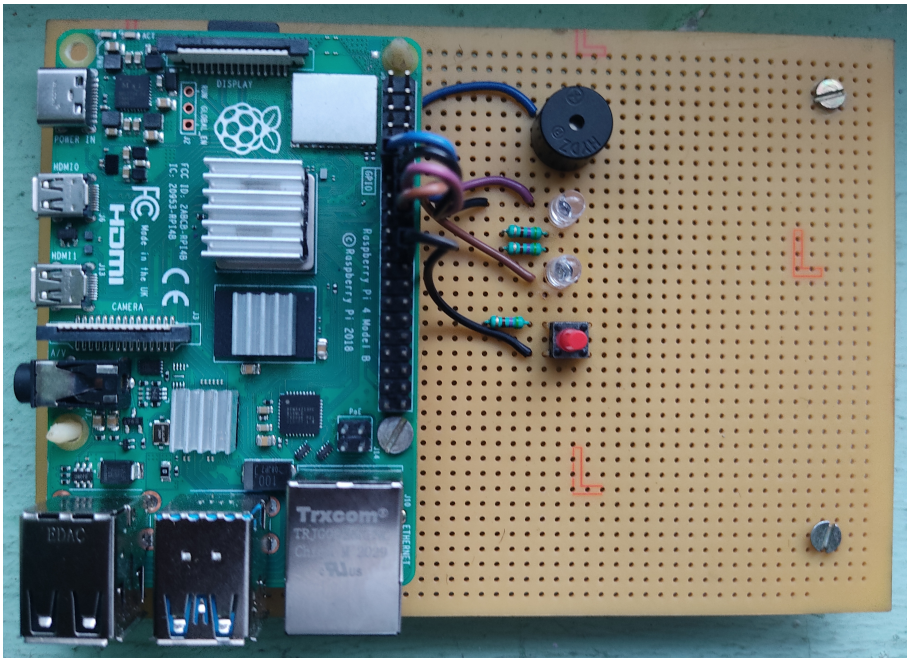


Figure 4.5: Prototype Model.

Chapter 5

EXPERIMENTS RESULTS AND DISCUSSION

The experimental results obtained in this project work is discussed here. The results are analyzed at various levels. The face mask detection python file is runned in a Raspberry Pi 4 module along with the YOLOv3 files so the images are feeded and the identification of persons wearing and not wearing masks is processed. figure 5.1 and figure 5.2 shows input images feeded to Raspberry Pi. Incase of live stream input video is received and they are processed frame by frame.

Figure 5.1: Input Image.

Figure 5.2: Input Image.

Here after detection, based on status the LED and Buzzer will glow. Table 5.1 given below shows the LED and buzzer activation scheme.

Table 5.1: LED and buzzer activation scheme

Status	Green LED	Red LED	Buzzer
Safe	ON	OFF	OFF
Warning	OFF	ON(Blink)	ON(Blink)
Danger	OFF	ON	ON

Once the detection is done then the situation or condition is displayed whether safe, warning or danger. Based upon the condition intimation is sent to the authority person through the IOT and the indication light glow and buzzer is activated. figure 5.3 and figure 5.4 show the output images.

Figure 5.3: Output Image.

Figure 5.4: Output Image.

After detection of persons with and without face masks the intimation message is sent to the authority person by means of blynk server. From the Raspberry Pi the mobile notification and the email is sent to the authority person. The mobile notification is received in Blynk mobile application. figure 5.5 and figure 5.6 shows the mobile notification and mail received. The mail and mobile notification consist of status and count of persons with mask and without mask.

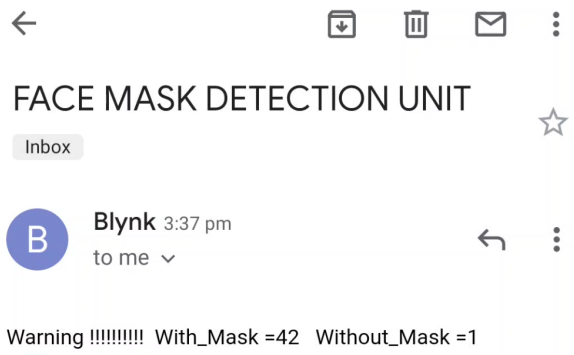


Figure 5.5: Mail Received.

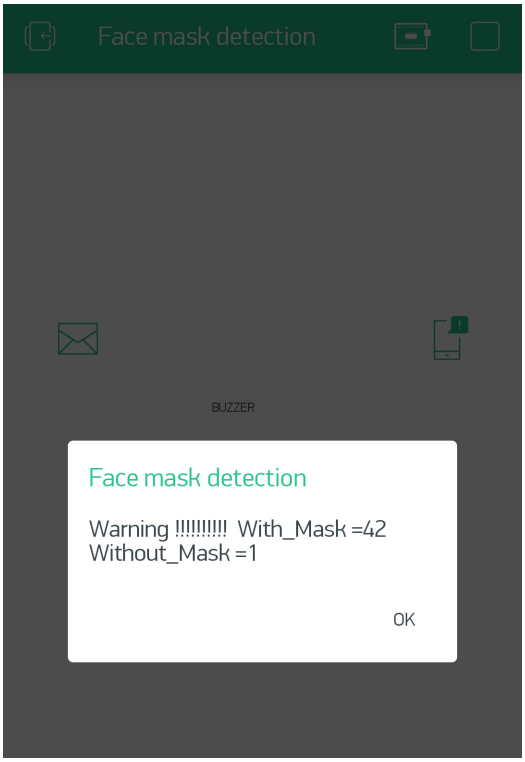


Figure 5.6: Mobile Notification.

Chapter 6

CONCLUSION AND FUTURE WORKS

In this work of face mask detection we have used YOLOv3 to detect the persons with face mask and without face mask with good efficiency and and sent an intimation message to authority persons by means of IOT. It's performance is really well in images and our detection results were also quite good. This detection can also be used for video stream or camera fed inputs. To get improved performance and speed, Raspberry Pi of higher variant such as 4GB or 8GB RAM can be used to implement the detection algorithm. The Future development of the project is planned to involve the identification of a person and sent the intimation message to the persons mobile who were not wearing face masks. This can be implemented in offices and institutions by means of training the database with employees images or students images and by means of face recognition the person is identified by which the mobile number and other details of the person is obtained from database and hence it will be easy to notify that particular person or useful for taking any actions regarding not wearing face mask.The proposed model can also be enhanced by means of including various parameters like peoples count, social distance and temperature measurement. This project will be very helpful and can be implemented in hospitals, airports, schools, colleges, offices, shops, malls, theaters, temples, apartments etc. and can also be implemented for Covid free event management.

APPENDIX - 1

7.1 PYTHON PROGRAM

About Python

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. It was created by Guido van Rossum during 1985- 1990. Like Perl, Python source code is also available under the GNU General Public License (GPL). Python 3.0 was released in 2008. Although this version is supposed to be backward incompatible, later on many of its important features have been back-ported to be compatible with version 2.7.

Features

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently whereas other languages use punctuation, and it has fewer syntactic constructions than other languages.

Libraries:

Python's standard library is very extensive, offering a wide range of facilities as indicated by the long table of contents listed below. The library contains built-in modules (written in C) that provide access to system functionality such as file I/O that would otherwise be inaccessible to Python programmers, as well as modules written in Python that provide standardized solutions for many problems that occur in everyday programming. Some of these modules are explicitly designed to encourage and enhance the portability of Python programs by abstracting away platform-specifics into platform-neutral APIs.

The Python installers for the Windows platform usually include the entire standard library and often also include many additional components. For Unix-like operating systems Python is normally provided as a collection of packages, so it may be necessary to use the packaging tools provided with the operating system to obtain some or all of the optional components.

Python program for Face Mask Detection of image and Sending mobile and Email notifications.

```
import numpy as np
import argparse
import time
import blynklib
import cv2
import os
import RPi.GPIO as GPIO # Import Raspberry Pi GPIO library
from time import sleep # Import the sleep function from the time module
GPIO.setwarnings(False) # Ignore warning for now
GPIO.setmode(GPIO.BCM) # Use physical pin numbering
GPIO.setup(23, GPIO.OUT, initial=GPIO.LOW)
GPIO.setup(24, GPIO.OUT, initial=GPIO.LOW)
GPIO.setup(17, GPIO.OUT, initial=GPIO.LOW)
GPIO.setup(25, GPIO.IN , pull_up_down=GPIO.PUD_UP)
flag=0
BLYNK_AUTH ='m-eIBHw6vDu47oED-QS7ozyXV2tctdSu'
TARGET_EMAIL = 'manomu29@gmail.com'
blynk = blynklib.Blynk(BLYNK_AUTH)
EMAIL_PRINT_MSG = "Email and Mobile Notification was sent"

ap = argparse.ArgumentParser()
ap.add_argument("-i", "--image", required=True, help="path to input image")
ap.add_argument("-o", "--output",help="path to output image")
ap.add_argument("-y", "--yolo", required=True, help="base path to YOLO directory")
ap.add_argument("-c", "--confidence", type=float, default=0.45,help="minimum
probability to filter weak detections")
ap.add_argument("-t", "--threshold", type=float, default=0.3,help="threshold when
applying non-max suppression")
args = vars(ap.parse_args())

labelsPath = os.path.sep.join([args["yolo"], "obj.names"])
LABELS = open(labelsPath).read().strip().split("\n")

COLORS = [[0,0,255],[0,255,0]]


weightsPath = os.path.sep.join([args["yolo"], "yolov3_face_mask.weights"])
configPath = os.path.sep.join([args["yolo"], "yolov3.cfg"])
```

```
print("[INFO] loading YOLO from disk...")
net = cv2.dnn.readNetFromDarknet(configPath, weightsPath)

image = cv2.imread(args["image"])
(H, W) = image.shape[:2]


ln = net.getLayerNames()
ln = [ln[i[0] - 1] for i in net.getUnconnectedOutLayers()]

blob = cv2.dnn.blobFromImage(image, 1 / 255.0, (832, 832),swapRB=True, crop=False)
net.setInput(blob)
start = time.time()
layerOutputs = net.forward(ln) #list of 3 arrays, for each output layer.
end = time.time()
print("[INFO] YOLO took {:.6f} seconds".format(end - start))

boxes = []
confidences = []
classIDs = []

for output in layerOutputs:
        for detection in output:
                scores = detection[5:] #last 2 values in vector
                classID = np.argmax(scores)
                confidence = scores[classID]
                if confidence > args["confidence"]:
                        box = detection[0:4] * np.array([W, H, W, H])
                        (centerX, centerY, width, height) = box.astype("int")
                        x = int(centerX - (width / 2))
                        y = int(centerY - (height / 2))
                        boxes.append([x, y, int(width), int(height)])
                        confidences.append(float(confidence))
                        classIDs.append(classID)

idxs = cv2.dnn.NMSBoxes(boxes, confidences, args["confidence"],args["threshold"])
border_size=100
border_text_color=[255,255,255]
image = cv2.copyMakeBorder(image, border_size,0,0,0, cv2.BORDER_CONSTANT)
filtered_classids=np.take(classIDs,idxs)
mask_count=(filtered_classids==1).sum()
nomask_count=(filtered_classids==0).sum()
text = "NoMaskCount: {}  MaskCount: {}".format(nomask_count, mask_count)
cv2.putText(image,text, (0, int(border_size-50)),
cv2.FONT_HERSHEY_SIMPLEX,0.8,border_text_color, 2)

text = "Status:"
cv2.putText(image,text, (W-300, int(border_size-50)),
```

```
cv2.FONT_HERSHEY_SIMPLEX,0.8,border_text_color, 2)
ratio=nomask_count/(mask_count+nomask_count)

if ratio>=0.1 and nomask_count>=3:
    text = "Danger !"
    cv2.putText(image,text, (W-200, int(border_size-50)),
cv2.FONT_HERSHEY_SIMPLEX,0.8,[26,13,247], 2)
    mss="{}!!!!!!!!withmask ={} withoutmask ={}".format(text,mask_count,nomask_count)
    flag=2
    @blynk.handle_event("connect")
    def connect_handler():
     print('Sleeping 2 sec before sending email...')
     time.sleep(2)
     blynk.email(TARGET_EMAIL, 'FACE MASK DETECTION UNIT', mss)
     blynk.notify(mss)
     print(EMAIL_PRINT_MSG)
     blynk.run()
elif ratio!=0 and np.isnan(ratio)!=True:
    text = "Warning !"
    cv2.putText(image,text, (W-200, int(border_size-50)),
cv2.FONT_HERSHEY_SIMPLEX,0.8,[0,255,255], 2)
    mss="{}!!!!!!!!!  With_Mask ={}    Without_Mask
={}".format(text,mask_count,nomask_count)
    flag=1
    @blynk.handle_event("connect")
    def connect_handler():
     print('Sleeping 2 sec before sending email...')
     time.sleep(2)
     blynk.email(TARGET_EMAIL, 'FACE MASK DETECTION UNIT', mss)
     blynk.notify(mss)
     print(EMAIL_PRINT_MSG)
     blynk.run()
else:
    text = "Safe "
    cv2.putText(image,text, (W-200, int(border_size-50)),
cv2.FONT_HERSHEY_SIMPLEX,0.8,[0,255,0], 2)
    flag=0

if len(idxs) > 0:

        for i in idxs.flatten():
                (x, y) = (boxes[i][0], boxes[i][1]+border_size)
                (w, h) = (boxes[i][2], boxes[i][3])
                color = [int(c) for c in COLORS[classIDs[i]]]
                cv2.rectangle(image, (x, y), (x + w, y + h), color, 1)
                text = "{}: {:.4f}".format(LABELS[classIDs[i]], confidences[i])
                cv2.putText(image, text, (x, y-5), cv2.FONT_HERSHEY_SIMPLEX,0.5,
color, 1)
if args["output"]:
```

```
            cv2.imwrite(args["output"],image)
            print("OUTPUT")
cv2.imshow("Image",image)
blynk.run()
if flag==1:
    while True:
   while True: # Run forever
        GPIO.output(23, GPIO.HIGH) # Turn on
        sleep(1) # Sleep for 1 second
        GPIO.output(17, GPIO.LOW) # Turn off
        sleep(1) # Sleep for 1 second
        GPIO.output(23, GPIO.LOW) # Turn off
        sleep(1) # Sleep for 1 second
        GPIO.output(17, GPIO.HIGH) # Turn on
        sleep(1) # Sleep for 1 second
        blynk.run()
        cv2.waitKey(50)
        if GPIO.input(25)==GPIO.LOW:
          print("Finished")
          break

if flag==2:
    while True: # Run forever
        GPIO.output(23, GPIO.HIGH) # Turn on
        GPIO.output(17, GPIO.HIGH) # Turn on
        cv2.waitKey(50)
        if GPIO.input(25)==GPIO.LOW:
          print("Finished")
          break
if flag==0:
    while True: # Run forever
        GPIO.output(24, GPIO.HIGH) # Turn on
        cv2.waitKey(50)
        if GPIO.input(25)==GPIO.LOW:
          print("Finished")
          break

GPIO.cleanup()
```

Above python program is for detection of persons with face mask and without face mask in an image which is feeded as input which is passed through command line argument. For detection of faces with and without mask in a video stream or webcam program which is given below is used. Here the video stream is captured by means of using openCv command and processed frame by frame.

Python Program for Face Mask Detection in video stream and Sending Email and mobile notification.

```
from imutils.video import FPS
import numpy as np
import argparse
import cv2
import os
import time
import blynklib
import RPi.GPIO as GPIO
from time import sleep

flag=0
BLYNK_AUTH ='m-eIBHw6vDu47oED-QS7ozyXV2tctdSu'
TARGET_EMAIL = 'manomu29@gmail.com'
blynk = blynklib.Blynk(BLYNK_AUTH)
EMAIL_PRINT_MSG = "Email and Mobile Notification was sent"

ap = argparse.ArgumentParser()
ap.add_argument("-y", "--yolo", required=True,help="base path to YOLO directory")
ap.add_argument("-i", "--input", type=str, default="",help="path to (optional) input video
file")
ap.add_argument("-o", "--output", type=str, default="",help="path to (optional) output
video file")
ap.add_argument("-d", "--display", type=int, default=1,help="whether or not output frame
should be displayed")
ap.add_argument("-c", "--confidence", type=float, default=0.45,help="minimum
probability to filter weak detections")
ap.add_argument("-t", "--threshold", type=float, default=0.3,help="threshold when
applyong non-maxima suppression")
ap.add_argument("-u", "--use-gpu", type=bool, default=0,help="boolean indicating if
CUDA GPU should be used")
args = vars(ap.parse_args())

labelsPath = os.path.sep.join([args["yolo"], "obj.names"])
LABELS = open(labelsPath).read().strip().split("\n")

COLORS = [[0,0,255],[0,255,0]]

weightsPath = os.path.sep.join([args["yolo"], "yolov3_face_mask.weights"])
configPath = os.path.sep.join([args["yolo"], "yolov3.cfg"])


print("[INFO] loading YOLO from disk...")
net = cv2.dnn.readNetFromDarknet(configPath, weightsPath)

ln = net.getLayerNames()
ln = [ln[i[0] - 1] for i in net.getUnconnectedOutLayers()]
```

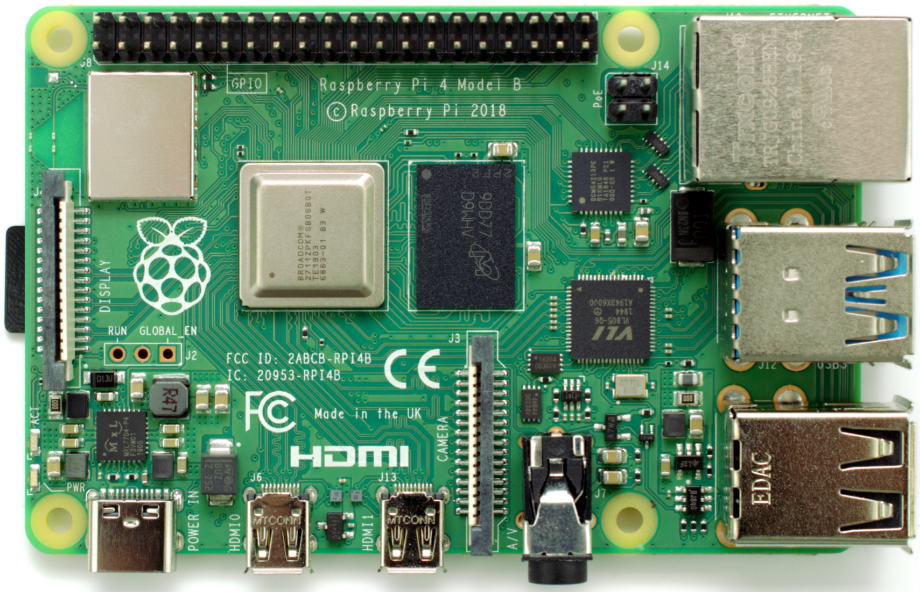
```
W = None
H = None
print("[INFO] accessing video stream...")
vs = cv2.VideoCapture(args["input"] if args["input"] else 0)
fps = FPS().start()

while True:
   GPIO.setwarnings(False) # Ignore warning for now
   GPIO.setmode(GPIO.BCM) # Use physical pin numbering
   GPIO.setup(23, GPIO.OUT, initial=GPIO.LOW)
   GPIO.setup(24, GPIO.OUT, initial=GPIO.LOW)
   GPIO.setup(17, GPIO.OUT, initial=GPIO.LOW)
   GPIO.setup(25, GPIO.IN , pull_up_down=GPIO.PUD_UP)
   (grabbed, frame) = vs.read()
  if not grabbed:
      break
   print("[INFO] Input Frame...")
   cv2.imshow("Input Frame", frame)
   cv2.waitKey(50)
  if W is None or H is None:
      (H, W) = frame.shape[:2]
  print("[INFO] Passing to YOLO detector...")
   blob = cv2.dnn.blobFromImage(frame, 1 / 255.0, (864, 864),swapRB=True, crop=False)
   net.setInput(blob)
   layerOutputs = net.forward(ln)
   boxes = []
   confidences = []
   classIDs = []

   for output in layerOutputs:
      for detection in output:
        scores = detection[5:]
         classID = np.argmax(scores)
         confidence = scores[classID]

         if confidence > args["confidence"]:
            box = detection[0:4] * np.array([W, H, W, H])
            (centerX, centerY, width, height) = box.astype("int")

            x = int(centerX - (width / 2))
            y = int(centerY - (height / 2))

            boxes.append([x, y, int(width), int(height)])
            confidences.append(float(confidence))
            classIDs.append(classID)

   idxs = cv2.dnn.NMSBoxes(boxes, confidences, args["confidence"],args["threshold"]
   border_size=100
```

```
    border_text_color=[255,255,255]
    frame = cv2.copyMakeBorder(frame, border_size,0,0,0, cv2.BORDER_CONSTANT)

    filtered_classids=np.take(classIDs,idxs)
    mask_count=(filtered_classids==1).sum()
    nomask_count=(filtered_classids==0).sum()

    text = "NoMaskCount: {}  MaskCount: {}".format(nomask_count, mask_count)
    cv2.putText(frame,text, (0, int(border_size-50)),
cv2.FONT_HERSHEY_SIMPLEX,0.65,border_text_color, 2)

    text = "Status:"
    cv2.putText(frame,text, (W-200, int(border_size-50)),
cv2.FONT_HERSHEY_SIMPLEX,0.65,border_text_color, 2)
    ratio=nomask_count/(mask_count+nomask_count+0.000001)


    if ratio>=0.1 and nomask_count>=3:
        text = "Danger !"
        #cv2.putText(frame,text, (W-100, int(border_size-50)),
cv2.FONT_HERSHEY_SIMPLEX,0.65,[26,13,247], 2)
        mss="{}!!!!!!!!withmask ={} withoutmask
={}".format(text,mask_count,nomask_count)
        flag=2
        @blynk.handle_event("connect")
        def connect_handler():
         print('Sleeping 2 sec before sending email...')
         time.sleep(2)
         blynk.email(TARGET_EMAIL, 'FACE MASK DETECTION UNIT', mss)
         blynk.notify(mss)
         print(EMAIL_PRINT_MSG)
         blynk.run()
    elif ratio!=0 and np.isnan(ratio)!=True:
        text = "Warning !"
        #cv2.putText(frame,text, (W-100, int(border_size-50)),
cv2.FONT_HERSHEY_SIMPLEX,0.65,[0,255,255], 2)
        mss="{}!!!!!!!!!  With_Mask ={}   Without_Mask
={}".format(text,mask_count,nomask_count)
        flag=1
        @blynk.handle_event("connect")
        def connect_handler():
         print('Sleeping 2 sec before sending email...')
         time.sleep(2)
         blynk.email(TARGET_EMAIL, 'FACE MASK DETECTION UNIT', mss)
         blynk.notify(mss)
         print(EMAIL_PRINT_MSG)
         blynk.run()
    else:
        text = "Safe "
```

```
        cv2.putText(frame,text, (W-100, int(border_size-50)),
cv2.FONT_HERSHEY_SIMPLEX,0.65,[0,255,0], 2)
        flag=0
    if len(idxs) > 0:
        for i in idxs.flatten():
            (x, y) = (boxes[i][0], boxes[i][1]+border_size)
            (w, h) = (boxes[i][2], boxes[i][3])
            color = [int(c) for c in COLORS[classIDs[i]]]
            cv2.rectangle(frame, (x, y), (x + w, y + h), color, 1)
            text = "{}: {:.4f}".format(LABELS[classIDs[i]], confidences[i])
            cv2.putText(frame, text, (x, y-5), cv2.FONT_HERSHEY_SIMPLEX,0.5, color, 1)

    cv2.imshow("Output Frame", frame)
    cv2.waitKey(50)
    print("[INFO] Output Frame...")
    fps.update()
     blynk.run()
    if flag==1:
        while True:
         GPIO.output(23, GPIO.HIGH) # Turn on
         sleep(1) # Sleep for 1 second
         GPIO.output(17, GPIO.LOW) # Turn off
         sleep(1) # Sleep for 1 second
         GPIO.output(23, GPIO.LOW) # Turn off
         sleep(1) # Sleep for 1 second
         GPIO.output(17, GPIO.HIGH) # Turn on
         sleep(1) # Sleep for 1 second
         if GPIO.input(25)==GPIO.LOW:
            print("Finished")
            GPIO.cleanup()
            break
    if flag==2:
        while True: # Run forever
         GPIO.output(23, GPIO.HIGH) # Turn on
         GPIO.output(17, GPIO.HIGH) # Turn on
         if GPIO.input(25)==GPIO.LOW:
            print("Finished")
            GPIO.cleanup()
            break
    if flag==0:
        GPIO.output(24, GPIO.HIGH) # Turn on
        time.sleep(2)
fps.stop()
print("[INFO] elasped time: {:.2f}".format(fps.elapsed()))
print("[INFO] approx. FPS: {:.2f}".format(fps.fps()))
```

7.2 RASPBERRY Pi 4

RASPBERRY PI HARDWARE DETAILS



Introduction:

Raspberry Pi 4 Model B is the latest product in the popular Raspberry Pi range of computers. It offers ground-breaking increases in processor speed, multimedia performance, memory, and connectivity compared to the prior-generation Raspberry Pi 3 Model B+, while retaining backwards compatibility and similar power consumption. For the end user, Raspberry Pi 4 Model B provides desktop performance comparable to entry-level x86 PC systems. This product's key features include a high-performance 64-bit quad-core processor, dual-display support at resolutions up to 4K via a pair of micro-HDMI ports, hardware video decode at up to 4Kp60, up to 8GB of RAM, dual-band 2.4/5.0 GHz wireless LAN, Bluetooth 5.0, Gigabit Ethernet, USB 3.0, and PoE capability (via a separate PoE HAT add-on). The dual-band wireless LAN and Bluetooth have modular compliance certification, allowing the board to be designed into end products with significantly reduced

compliance testing, improving both cost and time to market. The Pi4B requires a good quality USB-C power supply capable of delivering 5V at 3A. If attached downstream USB devices consume less than 500mA, a 5V, 2.5A supply may be used.

Pin Details

The Pi4B makes 28 BCM2711 GPIOs available via a standard Raspberry Pi 40-pin header. This header is backwards compatible with all previous Raspberry Pi boards with a 40-way header.

Table 2: Raspberry Pi 4 pin configuration

PIN GROUP	**PIN NAME**	**DESCRIPTION**
POWER SOURCE	+5V, +3.3V, GND and Vin	+5V -power output +3.3V -power output GND – GROUND pin
COMMUNICATION INTERFACE	UART Interface(RXD, TXD) [(GPIO15,GPIO14)]	UART (Universal Asynchronous Receiver Transmitter) used for interfacing sensors and other devices.
	SPI Interface (MOSI, MISO, CLK, CE) x 2 [SPI0-(GPIO10 ,GPIO9, GPIO11 ,GPIO8)] [SPI1--(GPIO20 ,GPIO19, GPIO21 ,GPIO7)]	SPI (Serial Peripheral Interface) used for communicating with other boards or peripherals.
	TWI Interface (SDA, SCL) x 2 [(GPIO2, GPIO3)] [(ID_SD,ID_SC)]	TWI (Two Wire Interface) Interface can be used to connect peripherals.
INPUT OUTPUT PINS	26 I/O	Although these pins have multiple functions they can be considered as I/O pins.
PWM	Hardware PWM available on GPIO12, GPIO13, GPIO18, GPIO19	These 4 channels can provide PWM (Pulse Width Modulation) outputs. *Software PWM available on all pins
EXTERNAL INTERRUPTS	All I/O	In the board all I/O pins can be used as Interrupts.

Raspberry Pi 4 Technical Specifications

Microprocessor	quad-core Arm Cortex-A72
Processor Operating Voltage	3.3V
Raw Voltage input	5V, 3A power source
Maximum current through each I/O pin	16mA
Maximum total current drawn from all I/O pins	50mA
Flash Memory	16Gbytes SSD memory card
Internal RAM	2Gbytes LPDDR4-3200 SDRAM
Clock Frequency	1.5GHz
GPU	Broadcom VideoCore VI
Wireless Connectivity	2.4 GHz and 5 GHz 802.11b/g/n/ac wireless LAN
Operating Temperature	-40ºC to +85ºC
USB	2 × USB 2.0, 2 × USB 3.0
Audio Output	3.5mm Jack and HDMI
Video output	HDMI
Camera Connector	15-pin MIPI Camera Serial Interface (CSI-2)
Memory Card Slot	Push/Pull Micro SDIO

Pi Processor

This is the Broadcom chip used in the Raspberry Pi 4 Model B. The architecture of the BCM2711 is a considerable upgrade on that used by the SoCs in earlier Pi models. It continues the quad-core CPU design of the BCM2837, but uses the more powerful ARM A72 core. It has a greatly improved GPU feature set with much faster input/output, due to the

incorporation of a PCIe link that connects the USB 2 and USB 3 ports, and a natively attached Ethernet controller. It is also capable of addressing more memory than the SoCs used before. The ARM cores are capable of running at up to 1.5 GHz, making the Pi 4 about 50% faster than the Raspberry Pi 3B+. The new VideoCore VI 3D unit now runs at up to 500 MHz. The ARM cores are 64-bit, and while the VideoCore is 32-bit, there is a new Memory Management Unit, which means it can access more memory than previous versions. The BCM2711 chip continues to use the heat spreading technology started with the BCM2837B0, which provides better thermal management.

7.3 IOT SERVER

webIOPi

There's a lot of handy apps and tools out there, but since we want to make something ourselves, this time we'll be using "WebIOPi" to operate GPIO with a browser.

WebIOPi is a software used to materialize the "IoT (Internet of Things)" with Raspberry Pi. It looks like it was published on Google Code before. **IOT** (Internet of Things), the Internet of Things (IOT) is the network of physical objects—devices, vehicles, buildings and other items—embedded with electronics, software, sensors, and network connectivity that enables these objects to collect and exchange data.

In WebIOPi's case, input and output are performed with a browser. You can operate GPIO with a browser button, and likewise acquire values from GPIO and display them quite easily.

weaved

Weaved services connect you easily and securely to your Pi from a mobile app or browser window. Control remote computers using tcp hosts such as ssh (remote terminal) and VNC (Virtual Network Console).

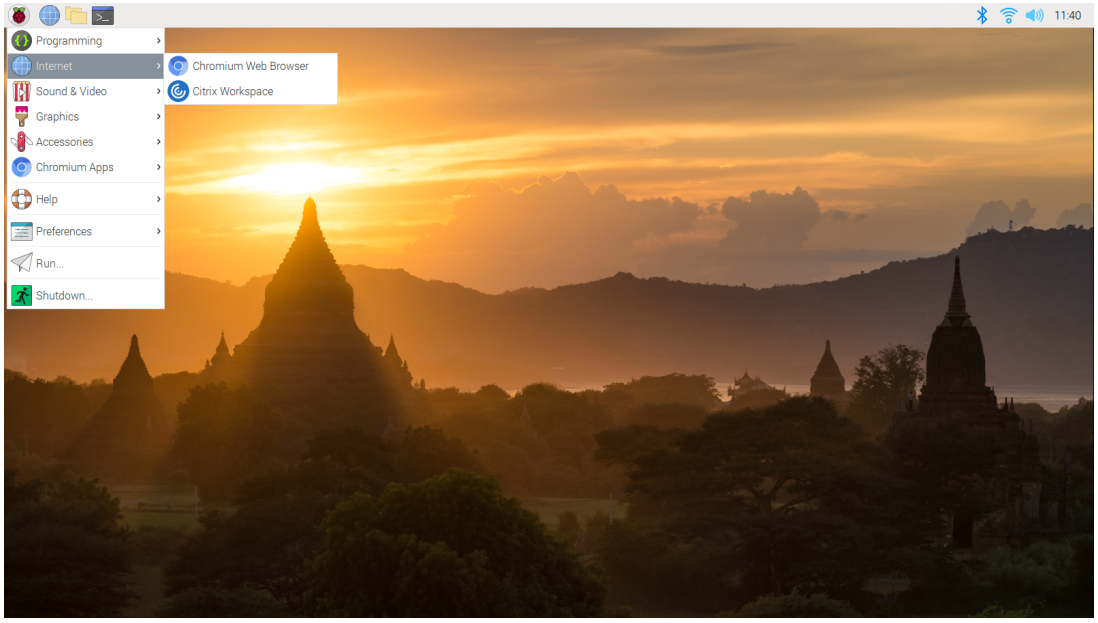
The easiest way we've seen to open up your Raspberry Pi as an Internet of Things device is to use the service Weaved. Weaved provides an IoT (Internet of Things) Kit for the Raspberry Pi. The kit provides really simple tools for connecting your Pi to the cloud, receiving notifications, and turning your Pi into an Internet of Things Kit.

BLYNK server

Blynk is an Internet of Things Platform aimed to simplify building mobile and web applications for the Internet of Things. Easily connect 400+ hardware models like Arduino, ESP8266, ESP32, Raspberry Pi and similar MCUs and drag-n-drop IOT mobile apps for iOS and Android in 5 minutes. Blynk Library can connect any hardware over Ethernet, WiFi, or GSM, 2G, 3G, LTE, etc. Extensive hardware-cloud-app API. Choose: C++, JS, Python, or HTTP. Blynk Cloud is open-source. It can be run by us, in your cloud environment like Amazon, or privately hosted on your local machine. Blynk Server is deployable in minutes. It's real-time, and ready to manage billions of requests from your edge devices.

7.4 VNC Viewer

Interfacing Raspberry Pi Using VNC Viewer



Virtual Network Computing (VNC) allows us to control the desktop of another computer remotely. This is especially useful for working with a Raspberry Pi if we don't want to have a monitor, keyboard, and mouse for each Raspberry Pi. The details to install and setup a VNC server on Raspbian and configure a VNC client on our computer is given below.

VNC Connect from RealVNC is included with Raspbian. It consists of both VNC Server, which allows us to control your Raspberry Pi remotely, and VNC Viewer, which allows us to control desktop computers remotely from our Raspberry Pi.

We must enable VNC Server before you can use it: instructions for this are given below. By default, VNC Server gives you remote access to the graphical desktop that is running on your Raspberry Pi.

However, we can also use VNC Server to gain graphical remote access to our Raspberry Pi if it is headless or not running a graphical desktop.

Enabling VNC Server

On your Raspberry Pi, run the following commands to make sure you have the latest version of VNC Connect:

sudo apt update

sudo apt install realvnc-vnc-server realvnc-vnc-viewer

Now enable VNC Server. You can do this graphically or at the command line.

Enabling VNC Server graphically

On your Raspberry Pi, boot into the graphical desktop.

Select **Menu > Preferences > Raspberry Pi Configuration > Interfaces**.

Ensure **VNC** is **Enabled**.

Enabling VNC Server at the command line

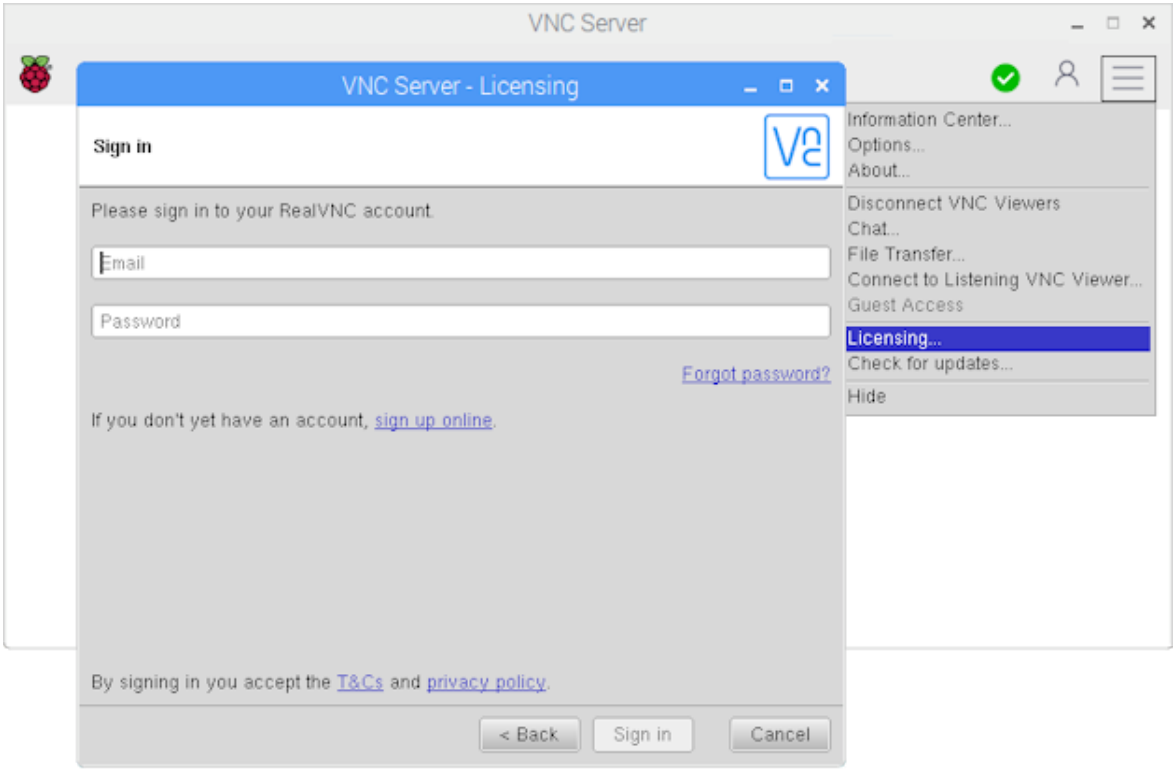
You can enable VNC Server at the command line using raspi-config:

sudo raspi-config

Now, enable VNC Server by doing the following:

Navigate to **Interfacing Options**.

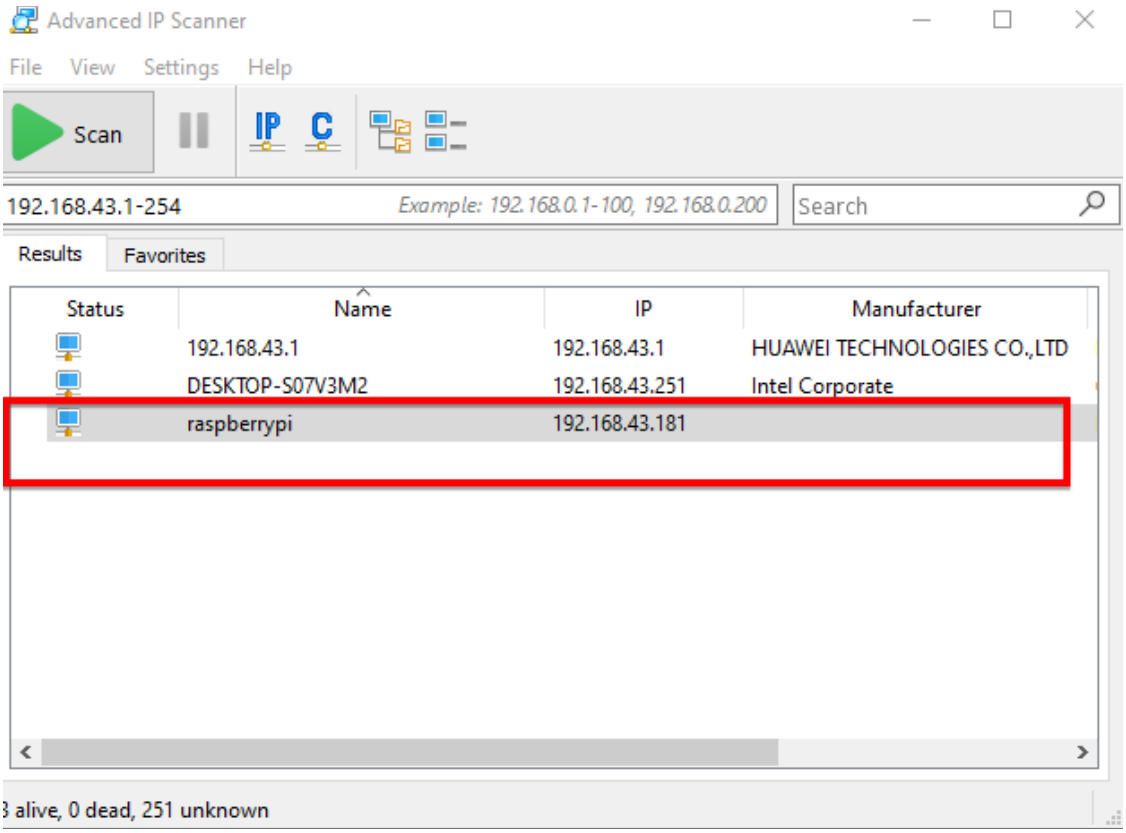
Scroll down and select **VNC > Yes**.



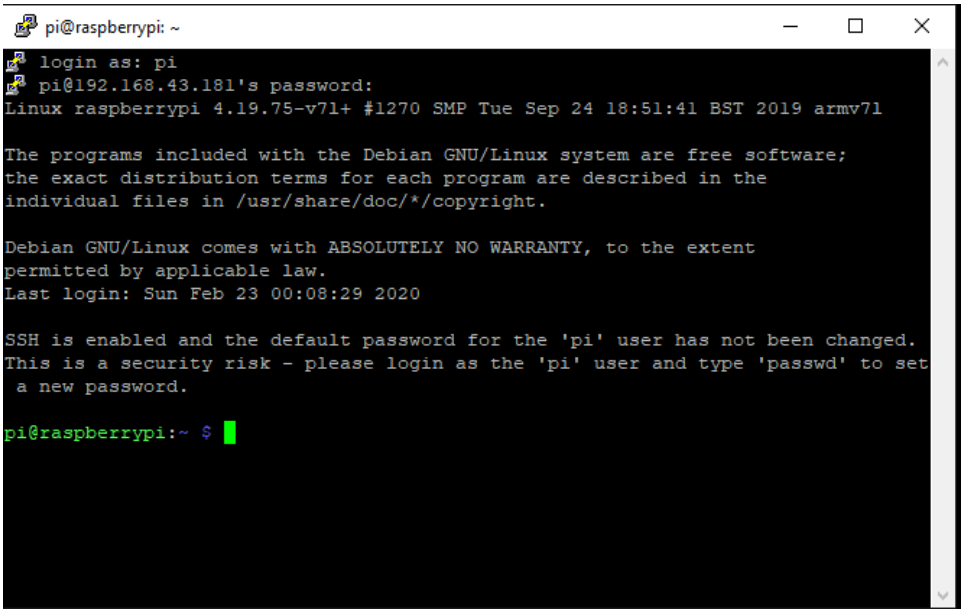
On the device we are going to take control, download VNC Viewer. we **must** use the compatible app from RealVNC. Sign in to VNC Viewer using the same RealVNC account credentials, and then either tap or click to connect to our Raspberry Pi.

Finding Raspberry Pi's IP Address

To find out the IP address of Raspberry Pi, we will use advanced IP Scanner. Download Advanced IP Scanner, install and open it.Press on the "Scan" button and it will show you the IP address of your Raspberry Pi along with other devices connected to this network. Note this IP address as we will need in the next step.

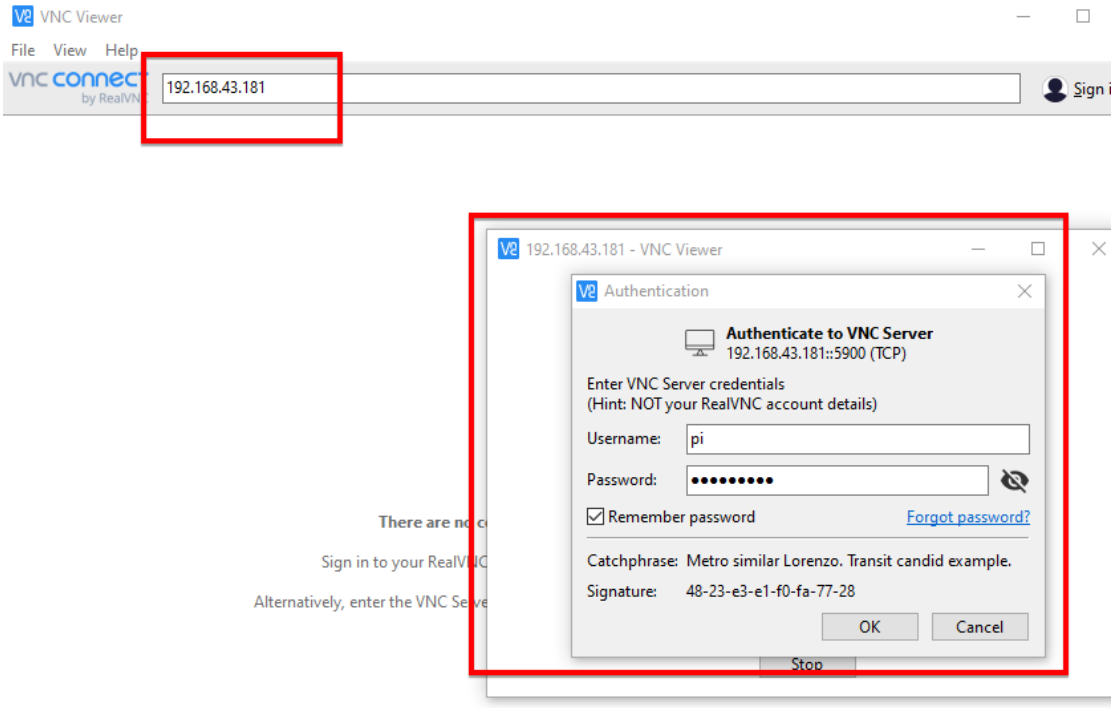


Open it, type the IP address of our Raspberry Pi (We got in the last step) and click on open.It will ask for username and password. Default username is 'pi' and password is 'raspberry'.



VNC Viewer on Client Side

Now we have to install a VNC viewer on our PC so that we can view and control it on the Raspberry Pi from our PC. Download and install VNC viewer and install it. Open it and type the IP address of your Raspberry Pi in it. It will ask for the username and password. The default username is 'pi' and the password is 'raspberry'.



Click on 'OK' and Finally, the Raspberry Pi desktop should appear as a VNC window. We will be able to access the GUI and do everything as if we were using the Pi's keyboard, mouse, and monitor directly.

APPENDICES - 2

CERTIFICATES:

This project has been submitted in "**NATIONAL WEB CONFERENCE ON CHALLENGES AND INNOVATION IN ENGINEERING AND TECHNOLOGY(NWCCIET-2021)**" organised by RAMCO INSTITUTE OF TECHNOLOGY and has won **BEST PAPER AWARD.**

RAMCO INSTITUTE OF TECHNOLOGY

Rajapalayam-626117

Approved by AICTE, New Delhi, Affiliated to Anna University, Chennai,
Accredited by NAAC & an ISO 9001:2015 Certified Institution

NATIONAL WEB CONFERENCE
ON
CHALLENGES AND INNOVATION IN ENGINEERING & TECHNOLOGY 2021
(In Association with Professional Societies)

Certificate of Presentation

This is to certify that Mr. M.U. Mano, Institute of Road and Transport technology, Erode has presented a paper entitled "Face Mask Detection and Warning System using AI and IoT" in the National Web Conference on Challenges and Innovation in Engineering & Technology 2021, during 19 - 20 March 2021.

Dr.S.Periyanayagi
Organising Secretary

Dr.S.Rajakarunakaran
Vice Principal

Dr.L.Ganesan
Principal

Certificate ID: NWCCIET21_ECE_05_04

REFERENCES

1. Ariyanto, Mochammad & Haryanto, Ismoyo & Setiawan, Joga & Muna, Munadi & Radityo, M.. (2019). Real-Time Image Processing Method Using Raspberry Pi for a Car Model. 46-51.

2. V. K. Bhanse and M. D. Jaybhaye,(2018) "Face Detection and Tracking Using Image Processing on Raspberry Pi," 2018 International Conference on Inventive Research in Computing Applications (ICIRCA), Coimbatore, India, pp. 1099-1103.

3. A. Das, M. Wasif Ansari and R. Basak, (2020) "Covid-19 Face Mask Detection Using TensorFlow, Keras and OpenCV," 2020 IEEE 17th India Council International Conference (INDICON), New Delhi, India, pp. 1-5.

4. M. S. Islam, E. Haque Moon, M. A. Shaikat and M. Jahangir Alam, (2020) "A Novel Approach to Detect Face Mask using CNN," 2020 3rd International Conference on Intelligent Sustainable Systems (ICISS), Thoothukudi, India, pp. 800-806.

5. Joseph Redmon, S. D. (2016) “You Only Look Once(YOLO) Unified, Real Time Object Detection” IEEE.

6. A. Lodh, U. Saxena, A. khan, A. Motwani, L. Shakkeera and V. Y. Sharmasth, (2020) "Prototype for Integration of Face Mask Detection and Person Identification Model – COVID-19," 2020 4th International Conference on Electronics, Communication and Aerospace Technology (ICECA), Coimbatore, India, pp. 1361-1367.

7. Luigi Atzori, Antonio iera and Giacomo Morabito. (2010) ‘The Internet of Things: A Survey’, Journal of Computer Networks Vol.54, No.15,pp.2787-2805.

8. Lu Tan and Neng Wang. (2010) ‘Future internet: The Internet of Things’, IEEE Xplore Proc., 3rd IEEE Int.Conf. Adv. Comp. Theory. Engg. (ICACTE), pp:1-9.

9. J. Marot and S. Bourennane, (2017)"Raspberry Pi for image processing education,"25th European Signal Processing Conference (EUSIPCO), Kos, Greece, 2017, pp. 2364-2366.

10. S. A. Sanjaya and S. Adi Rakhmawan, (2020) "Face Mask Detection Using MobileNetV2 in The Era of COVID-19 Pandemic," 2020 International Conference on Data Analytics for Business and Industry: Way Towards a Sustainable Economy (ICDABI), Sakheer, Bahrain, pp. 1-5.

11. Senthilkumar.R and Gnanamurthy.R.K. (2016) 'A Comparative Study of 2D PCA Face Recognition Method with Other Statistically Based Face Recognition Methods', Journal of the Institution of Engineers India Series B (Springer Journal), Vol.97, pp.425-430.

12. Senthilkumar.R and Gnanamurthy.R.K. (2017) 'Performance improvement in classification rate of appearance based statistical face recognition methods using SVM classifier", the IEEE International Conference on Advanced Computing and Communication Systems (ICACCS), 6-7 January 2017, pp. 286-292.

13. Senthilkumar.R and Gnanamurthy.R.K. (2018) 'HANFIS: A New Fast and Robust Approach for Face Recognition and Facial Image Classification', Advances in Intelligent Systems and Computing Smart Innovations in Communication and Computational Sciences, Chapter 8, pp:81-99.

14. S. Susanto, F. A. Putra, R. Analia and I. K. L. N. Suciningtyas, (2020) "The Face Mask Detection For Preventing the Spread of COVID-19 at Politeknik Negeri Batam," 2020 3rd International Conference on Applied Engineering (ICAE), Batam, Indonesia, pp. 1-5.

15. S. S. Walam, S. P. Teli, B. S. Thakur, R. R. Nevarekar and S. M. Patil, (2018) "Object Detection and seperation Using Raspberry PI," 2018 Second International Conference on Inventive Communication and Computational Technologies (ICICCT), Coimbatore, India, pp. 214-217.

16. Yair Meidan, Michael Bohadana, Asaf Shabatai, Juan David Guarnizo and NilsOle Tippenhauer and Yuval Elovici. (2017) 'ProfilloT: a machine learning approach for IoT device identification of network traffic analysis', proc.Sym. App. Comp., SAC, pp: 506-509.

17. G. Yang et al.,(2020) "Face Mask Recognition System with YOLOV5 Based on Image Recognition," 2020 IEEE 6th International Conference on Computer and Communications (ICCC), Chengdu, China, pp. 1398-1404.